Database Systems

Course Code: **CSC104**

COURSE INSTRUCTOR

Ms. Kainat Akram

REQUIRED: Following books are recommended:

- **B1:** Database Systems 2nd Edition by Garcia-Molina
- **B2:** First Course in Database Systems 3rd Edition by Ullman

OPTIONAL/OTHERS:

- **OB1:** Database Systems: The Complete Book (2nd edition) by Garcia-Molina, Ullman, and Widom)
- **OB2:** Database Management Systems (3rd edition) by Ramakrishnan and Gehrke (R/G)
- **OB3:** Fundamentals of Database Systems (6th edition) by Elmasri and Navathe (E/N)
- **OB4:** Database System Concepts (6th edition) by Silberschatz, Korth, and Sudarshan (S/K/S)

Lecture 12-13

11	Views and Indexes (HW # 3 out)	B2: 8.1, 8.2 and 8.3	**2**
12	Indexing and IO model Searching, Sorting and Hashing	B2: 8.4, 8.5	**4**

Chapter 8 – Topics at a Glance

- 8.1 Virtual Views — defining and querying virtual relations
- 8.2 Modifying Views — updatable views, DROP VIEW, instead-of triggers
- 8.3 Indexes in SQL — CREATE INDEX, DROP INDEX, motivation
- 8.4 Selection of Indexes — cost model, choosing the best index
- 8.5 Materialized Views — maintaining, periodic refresh, query rewriting

Section 8.1

Virtual Views

Relations, Tables, and Views

Term	Description
Table (Base Relation)	Stored physically in the database — persistent, modifiable via SQL
Virtual View	Defined by a query; not stored; computed on-the-fly when queried
Temporary Relation	Neither stored nor a view; exists only during a subquery execution

Key distinction: SQL programmers often say "table" for a stored relation and "view" for a virtual relation.

8.1.1 Declaring Views — Syntax

```
CREATE VIEW <view-name> AS <view-definition>;

-- view-definition is a standard SQL query

-- Optionally rename columns:
CREATE VIEW <view-name>(col1, col2, ...) AS <view-definition>;
```

The view is not stored. Its definition is saved and the query is re-executed whenever the view is used.

Example 8.1 — Simple View (ParamountMovies)

Example 8.1

Create a view showing title and year of movies made by Paramount Studios.

```
-- Base table:
Movies(title, year, length, genre, studioName, producerC#)

-- View Definition:
CREATE VIEW ParamountMovies AS
    SELECT title, year
    FROM Movies
    WHERE studioName = 'Paramount';
```

Result / Explanation:

```
-- View 'ParamountMovies' now contains:
-- All rows from Movies WHERE studioName = 'Paramount'
-- Only columns: title, year
-- It is NOT stored physically — defined and saved as a query
```

Example 8.2 — View with JOIN (MovieProd)

Example 8.2

Create a view 'MovieProd' that links movie titles to producer names (JOIN of two tables).

```
-- Base tables:
Movies(title, year, length, genre, studioName, producerC#)
MovieExec(name, address, cert#, netWorth)

CREATE VIEW MovieProd AS
    SELECT title, name
    FROM Movies, MovieExec
    WHERE producerC# = cert#;
```

Result / Explanation:

```
-- View contains: (title, name) pairs where certificate numbers match
-- This view joins two tables and shows which exec produced which movie
```

8.1.2 Querying Views

```
-- Views can be queried exactly like stored tables:

-- Example 8.3: Find Paramount movies from 1979
SELECT title
FROM ParamountMovies
WHERE year = 1979;

-- Example 8.4: Stars of all Paramount movies (view + base table)
SELECT DISTINCT starName
FROM ParamountMovies, StarsIn
WHERE title = movieTitle AND year = movieYear;
```

The DBMS replaces the view name with its definition as a subquery internally.

How DBMS Expands View Queries Internally

```
-- Original query using view:
SELECT DISTINCT starName
FROM ParamountMovies, StarsIn
WHERE title = movieTitle AND year = movieYear;

-- Expanded by DBMS (view replaced by subquery):
SELECT DISTINCT starName
FROM (SELECT title, year
      FROM Movies
      WHERE studioName = 'Paramount') Pm,
     StarsIn
WHERE Pm.title = movieTitle AND Pm.year = movieYear;
```

This expansion happens automatically — the programmer writes the simpler version.

8.1.3 Renaming Attributes in Views

```
-- Default: column names come from the SELECT clause
-- We can explicitly rename them:

CREATE VIEW MovieProd(movieTitle, prodName) AS
    SELECT title, name
    FROM Movies, MovieExec
    WHERE producerC# = cert#;

-- Now the view columns are named:
--   movieTitle  (instead of title)
--   prodName    (instead of name)
```

Useful when the original attribute names are ambiguous or you want a cleaner interface.

Section 8.2

Modifying Views

Can We Modify a View?

- A view has no physical storage — so what does it mean to INSERT/DELETE/UPDATE it?
- For complex views: the answer is usually 'you can't'.
- For simple views (called updatable views): modifications are translated to the underlying base table.
- Alternatively: 'INSTEAD OF' triggers can intercept any modification and redirect it to base tables.

8.2.1 View Removal — DROP VIEW

```
-- Remove a view definition:
DROP VIEW ParamountMovies;

-- This ONLY removes the view definition,
-- it does NOT affect the underlying Movies table.

-- But if we drop the base table:
DROP TABLE Movies;

-- Then ParamountMovies becomes unusable
-- (it references the now-nonexistent Movies table)
```

DROP VIEW removes the logical definition. DROP TABLE removes physical data AND breaks dependent views.

8.2.2 SQL Rules for Updatable Views

- A view is updatable in SQL if it satisfies ALL of these conditions:
- 1. Uses SELECT (not SELECT DISTINCT)
- 2. FROM clause references exactly ONE base relation R (no joins)
- 3. WHERE clause does NOT contain a subquery referencing R
- 4. SELECT clause includes enough attributes so other attrs can be filled with NULL/default
- 5. No attribute declared NOT NULL (without a default) is omitted from SELECT

- Modifications on updatable views are passed through to the underlying base table R.

Example 8.5 — INSERT into Updatable View

Example 8.5

Insert a new movie into the ParamountMovies view.

```
-- Attempt 1: Insert into view with only title, year
INSERT INTO ParamountMovies
VALUES('Star Trek', 1979);

-- DBMS translates to:
INSERT INTO Movies(title, year)
VALUES('Star Trek', 1979);
```

Result / Explanation:

```
-- Result: inserted tuple has NULL for length, genre, studioName, producerC#
-- Problem: studioName is NULL so it does NOT appear in ParamountMovies!

-- Fix: include studioName in view and insert:
CREATE VIEW ParamountMovies AS
    SELECT studioName, title, year FROM Movies WHERE studioName='Paramount';
INSERT INTO ParamountMovies VALUES('Paramount','Star Trek',1979);
-- Now studioName is set and the tuple appears correctly in the view
```

Example 8.6 — DELETE from Updatable View

Example 8.6

Delete all movies with 'Trek' in title from ParamountMovies view.

```
-- Delete from view:
DELETE FROM ParamountMovies
WHERE title LIKE '%Trek%';

-- DBMS translates to (adds view's WHERE condition):
DELETE FROM Movies
WHERE title LIKE '%Trek%'
  AND studioName = 'Paramount';
```

Result / Explanation:

```
-- The view's WHERE clause (studioName='Paramount') is automatically added
-- This ensures only Paramount movies with 'Trek' are deleted
-- Tuples from other studios are NOT affected
```

Example 8.7 — UPDATE through Updatable View

Example 8.7

Set year = 1979 for 'Star Trek the Movie' via the ParamountMovies view.

```
-- Update via view:
UPDATE ParamountMovies
SET year = 1979
WHERE title = 'Star Trek the Movie';

-- DBMS translates to:
UPDATE Movies
SET year = 1979
WHERE title = 'Star Trek the Movie'
  AND studioName = 'Paramount';
```

Result / Explanation:

```
-- Again, the view's condition is appended to the base table update
-- Only tuples visible in the view are affected
```

Why Some Views Are NOT Updatable

Consider view MovieProd (JOIN of Movies and MovieExec):

```
INSERT INTO MovieProd VALUES('Greatest Show on Earth', 'Cecil B. DeMille');
-- Problem: WHERE does this tuple go? Movies AND MovieExec both need a row.
```

Two-table joins are not updatable because:

- producerC# and cert# must be equal — but both would be NULL after insert
- SQL does not match NULL = NULL in a JOIN condition
- So the inserted row never appears in the view — the insert fails logically

8.2.3 Instead-Of Triggers on Views

- BEFORE / AFTER triggers fire around a normal event on a base table.
- INSTEAD OF triggers fire on a VIEW and replace the event entirely.
- When modification is attempted on the view:
- → The modification does NOT happen
- → The trigger's action runs in its place
- The designer writes the trigger to correctly update the underlying base tables.
- This gives full control over how any modification to a view is handled.

Example 8.8 — Instead-Of Trigger (INSERT into ParamountMovies)

Example 8.8

Write a trigger so that inserting into ParamountMovies correctly sets studioName = 'Paramount'.

```
CREATE TRIGGER ParamountInsert
INSTEAD OF INSERT ON ParamountMovies
REFERENCING NEW ROW AS NewRow
FOR EACH ROW
INSERT INTO Movies(title, year, studioName)
VALUES(NewRow.title, NewRow.year, 'Paramount');
```

Result / Explanation:

```
-- Line 2: INSTEAD OF means the insert into ParamountMovies NEVER happens
-- Line 3: NewRow refers to the tuple the user tried to insert
-- Lines 5-6: Instead, we insert into Movies with studioName hardcoded as
'Paramount'
-- Result: studioName is always correctly set, even though user didn't supply it
```

Section 8.3

Indexes in SQL

What is an Index?

- An index on attribute A of relation R is a data structure that allows fast lookup of tuples with a given value of A.
- Think of it as a binary search tree (key → set of disk locations of matching tuples).
- Useful for queries with: A = constant, A < constant, A > constant, range queries.
- The real implementation uses a B-tree (generalization of balanced binary tree) — covered in DBMS internals.
- The index key can be ANY attribute or set of attributes — it need NOT be the relation's primary key.
- Indexes are NOT part of the SQL standard but ALL commercial DBMS products support them.

8.3.1 Motivation for Indexes

```
SELECT * FROM Movies
WHERE studioName = 'Disney' AND year = 1990;
```

- Without index: Scan ALL 10,000 tuples of Movies — very slow!
- With index on year: Jump directly to the 200 tuples from 1990 — much faster.
- With index on (studioName, year): Jump directly to ~10 matching tuples — fastest!

Example 8.9 — Index Helping a JOIN Query

Example 8.9

Find the name of the producer of 'Star Wars'. Show how indexes speed up the JOIN.

```
SELECT name
FROM Movies, MovieExec
WHERE title = 'Star Wars'
  AND producerC# = cert#;
```

Without indexes: scan ALL tuples of both Movies and MovieExec.

Result / Explanation:

```
With index on Movies.title:
  → Find the one 'Star Wars' tuple in Movies quickly
  → Extract its producerC# value

With index on MovieExec.cert#:
  → Use producerC# to find the ONE matching MovieExec tuple
  → Extract the name
```

Result: Only 2 tuples examined (1 from each table) instead of scanning both

8.3.2 Declaring Indexes — Syntax

```
-- Create index on single attribute:
CREATE INDEX YearIndex ON Movies(year);

-- Create index on multiple attributes (composite key):
CREATE INDEX KeyIndex ON Movies(title, year);

-- Delete an index:
DROP INDEX YearIndex;

-- Note: index name must be given (used for DROP INDEX later)
```

After CREATE INDEX, the query processor automatically uses the index for matching queries. No query changes needed.

Example 8.10 — Multi-Attribute Index

Example 8.10

Create an index on (title, year) — the key for Movies. Understand ordering of attributes.

```
CREATE INDEX KeyIndex ON Movies(title, year);

-- (title, year) together form the key for Movies
-- A composite index on (title, year) can also be used
-- for queries that specify ONLY the first attribute: title
```

Result / Explanation:

```
-- If query specifies both title AND year: index finds exactly 1 tuple (key!)
-- If only title is specified: index still helps — finds all movies with that
title
-- If only year is specified: (title, year) index does NOT help efficiently

-- Ordering matters! If year queries are more common, create index on (year,
title)
-- CREATE INDEX YearTitleIdx ON Movies(year, title);
```

Section 8.4

Selection of Indexes

8.4 Index Selection — The Trade-Off

Benefits of Indexes

• Queries filtering by a value/range of the indexed attribute run much faster

• JOIN operations using the indexed attribute are faster

• Can avoid full table scans on large relations

Costs of Indexes

• Every INSERT, DELETE, UPDATE on the relation also requires updating the index

• Index is stored on disk — extra space used

• Modifications take ~2x longer because index page must be read AND written back

8.4.1 Simple Cost Model

- Tuples of a relation are stored across many disk pages.
- To examine even ONE tuple: the ENTIRE page must be loaded into main memory.
- Reading one page or all tuples on that page costs the same (one disk read).
- We assume pages are never already in memory — every access costs a disk read.
- Modification costs ~2x query (need to read the page, then write it back).
- Goal: Count the number of disk page accesses to estimate query/update cost.

8.4.2 Useful Indexes

- 1. Index on the KEY of a relation is almost always useful:
- → Key queries are common → At most ONE tuple matches → At most 1 page loaded

- 2. Index on a non-key attribute is useful if:
- (a) Attribute is 'almost a key' — very few tuples share a given value
- (b) Relation is CLUSTERED on that attribute (tuples with same value stored on same pages)

- Clustering: Group tuples with the same attribute value onto as few pages as possible.
- → Even if many tuples match, they are on few pages → fewer disk reads

Example 8.12 — Non-Key Index (title in Movies)

Example 8.12

Index on Movies.title (not a key — multiple movies can share a title like 'King Kong').

```
-- Querying for title = 'King Kong':
SELECT * FROM Movies WHERE title = 'King Kong';

-- 3 movies with that title exist (1933, 1976, 2005)
-- Index on title returns 3 locations
```

Result / Explanation:

```
-- 3 tuples → possibly 3 different pages → 3 disk reads
-- Without index: scan entire Movies relation (many pages)
-- With index: only ~3 page reads → significant saving even for non-key index

-- Also need producerC# from each → use index on MovieExec.cert# to find producers
-- Total: ~6 page reads vs. scanning both complete relations
```

Example 8.13 — Clustered vs. Non-Clustered Index

Example 8.13

Index on Movies.year — compare clustered vs. non-clustered storage.

```
SELECT * FROM Movies WHERE year = 1990;

-- Case 1: Tuples stored alphabetically by title (NOT clustered by year)
-- Case 2: Tuples CLUSTERED on year (all 1990 movies on same pages)
```

Result / Explanation:

```
Case 1 (Not clustered by year):
  → 1990 movies scattered across many pages
  → Index on year gives ~100 locations; each on different page
  → Must load most of the relation anyway → index gives little benefit

Case 2 (Clustered on year):
  → All 1990 movies are together on a few consecutive pages
  → Index finds the small page range → only those pages are loaded
  → Huge speed improvement!
```

Example 8.14 — Calculating Best Index (StarsIn)

Relation: StarsIn(movieTitle, movieYear, starName) | Size: 10 pages | Avg: 3 stars/movie, 3 movies/star

Action	No Index	Star Index	Movie Index	Both Indexes
Q1 (given star)	10	4	10	4
Q2 (given movie)	10	10	4	4
I (insertion)	2	4	4	6
Average	$2 + 8p_1 + 8p_2$	$4 + 6p_2$	$4 + 6p_1$	$6 - 2p_1 - 2p_2$

- p_1 = fraction of operations that are Q1 (query by star), p_2 = fraction that are Q2 (query by movie)
- If $p_1=p_2=0.1$ (mostly insertions): prefer NO index (cost = 2+0.8+0.8 = 3.6 minimum)
- If $p_1=p_2=0.4$ (many queries): prefer BOTH indexes (cost = 6−0.8−0.8 = 4.4 minimum)
- If $p_1=0.5$, $p_2=0.1$: prefer Star index only (cost = 4+0.6 = 4.6, best formula)

8.4.4 Automatic Index Selection (Tuning Advisors)

- Step 1: Establish query workload — examine DBMS log for typical queries and modifications.
- Step 2: Designer specifies constraints (indexes that must or must not be created).
- Step 3: Tuning advisor generates candidate indexes; query optimizer estimates running time for each.
- Step 4: Index set with lowest total cost is recommended (or automatically created).

- Greedy Algorithm for choosing multiple indexes:
- (a) Pick the single index with greatest benefit (if positive benefit exists).
- (b) Re-evaluate all remaining candidates assuming the chosen index is present.
- (c) Repeat until no candidate has positive benefit.

Section 8.5

Materialized Views

Materialized Views vs Virtual Views

Virtual View

- Defined by a query
- NOT stored — computed on demand
- Always reflects current data
- Free to create — no storage cost
- Slow for complex queries (recomputed each time)

Materialized View

- Also defined by a query
- IS stored — like a base table
- May be stale if base tables change
- Costs storage and maintenance
- Fast for complex queries (precomputed)
- Created with: CREATE MATERIALIZED VIEW

8.5 Declaring Materialized Views — Syntax

```
-- Standard syntax:
CREATE MATERIALIZED VIEW <view-name> AS
    <query-definition>;

-- Example (from Example 8.15):
CREATE MATERIALIZED VIEW MovieProd AS
    SELECT title, year, name
    FROM Movies, MovieExec
    WHERE producerC# = cert#;

-- MovieProd is now stored as a real table on disk
```

Unlike virtual views, the query is executed ONCE to populate the materialized view. Then it must be maintained.

Example 8.15 — Maintaining MaterializedView (MovieProd)

Base Table Change	Action on Materialized View MovieProd
INSERT new movie into Movies (title='Kill Bill', year=2003, producerC#=23456)	Lookup cert#=23456 in MovieExec → get name INSERT INTO MovieProd VALUES('Kill Bill',2003,'Quentin Tarantino')
DELETE movie from Movies (title='Dumb & Dumber', year=1994)	DELETE FROM MovieProd WHERE title='Dumb & Dumber' AND year=1994
INSERT new exec into MovieExec (cert#=34567, name='Max Bialystock')	Find all Movies with producerC#=34567 INSERT INTO MovieProd (title,year,'Max Bialystock') for each
DELETE exec from MovieExec (cert#=45678)	DELETE FROM MovieProd WHERE (title,year) IN (SELECT title,year FROM Movies WHERE producerC#=45678)

Key insight: All changes are INCREMENTAL — we never recompute the whole view from scratch.

8.5.2 Periodic Maintenance of Materialized Views

- Use case: Database serves two purposes simultaneously —
- (a) Recording current data (e.g., inventory with each sale = many modifications)
- (b) Analytics (studying buyer patterns = complex aggregation queries)

- Problem: If modifications are dominant, maintaining a materialized view on every change is too expensive.

- Solution: Reconstruct the materialized view PERIODICALLY (e.g., every night during low-activity period).
- → Analysts query the materialized view (possibly up to 24 hours stale)
- → Acceptable if the data changes slowly (e.g., sales trends don't change overnight)
- → Risk: sudden spikes (e.g., a celebrity wearing a product) won't be seen until next refresh

8.5.3 Rewriting Queries to Use Materialized Views

A query Q can use materialized view V if ALL conditions are met:

Condition	Meaning
1. FROM clause	All relations in V's FROM clause also appear in Q's FROM clause
2. WHERE clause	Q's WHERE = V's WHERE AND some additional condition C
3. C references V's attrs	Any attributes of V's FROM-relations mentioned in C are in V's SELECT list
4. SELECT attributes	All output attributes from Q that come from V's FROM-relations are in V's SELECT

Rewriting rule:

- (a) Replace V's relations in Q's FROM clause with V
- (b) Replace Q's WHERE clause with just the extra condition C

Example 8.16 — Query Rewriting Using Materialized View

Example 8.16

Use view MovieProd to simplify query: Find stars of movies produced by 'Max Bialystock'.

```
-- Original query (3-way join):
SELECT starName
FROM StarsIn, Movies, MovieExec
WHERE movieTitle=title AND movieYear=year
  AND producerC#=cert# AND name='Max Bialystock';

-- Materialized view MovieProd covers: Movies JOIN MovieExec ON producerC#=cert#
-- Condition C = movieTitle=title AND movieYear=year AND name='Max Bialystock'
```

Result / Explanation:

```
-- Rewritten query (2-way join using materialized view):
SELECT starName
FROM StarsIn, MovieProd
WHERE movieTitle = title
  AND movieYear = year
  AND name = 'Max Bialystock';

-- Movies and MovieExec replaced by MovieProd
-- producerC#=cert# condition removed (already encoded in MovieProd)
-- 2-way join is faster than 3-way join!
```

8.5.4 Automatic Creation of Materialized Views

- Similar process to automatic index selection (Section 8.4.4).
- Challenge: Infinite possible views — unlike indexes (one per attribute set).

- Limit candidates to views that:
- 1. Have FROM clause $\subseteq$ FROM clause of at least one workload query
- 2. Have WHERE clause = AND of conditions each appearing in at least one query
- 3. Have SELECT attributes sufficient to answer at least one query

- Benefit = improvement in average query execution time / space occupied by view
- → Materialized views can be much LARGER than the relations they are built on!
- → Size must factor into the benefit calculation (unlike indexes)

Chapter 8 Summary

Key Concepts Review

Chapter 8 — Summary Table

Concept	Key Points
Virtual Views	Defined by a query; not stored; queried like tables; DBMS expands at query time
Updatable Views	Single-relation views can accept INSERT/DELETE/UPDATE passed to base table
Instead-Of Triggers	Intercept view modifications and redirect to custom base-table operations
Indexes	Speed up queries on large relations; cost: slower modifications; syntax: CREATE INDEX
Index Selection	Trade-off: query speed vs. modification overhead; use cost model with p_1, p_2
Materialized Views	Stored result of a view query; fast to query; must be maintained when base tables change
Maintaining Mat. Views	Changes are incremental — only affected tuples updated, not full recompute
Periodic Refresh	For modification-heavy databases: rebuild materialized view nightly
Query Rewriting	Optimizer can replace parts of a query with a materialized view for speed

Practice Problems

Exercises with Solutions

Exercise 8.1.1 — Creating Views (a, b, c)

Base tables: MovieStar(name,address,gender,birthdate) MovieExec(name,address,cert#,netWorth) Studio(name,address,presC#)

```
-- (a) RichExec: executives with netWorth >= $10,000,000
CREATE VIEW RichExec AS
    SELECT name, address, cert#, netWorth
    FROM MovieExec WHERE netWorth >= 10000000;

-- (b) StudioPres: executives who are studio presidents
CREATE VIEW StudioPres AS
    SELECT name, address, cert#
    FROM MovieExec WHERE cert# IN (SELECT presC# FROM Studio);

-- (c) ExecutiveStar: individuals who are both exec AND star
CREATE VIEW ExecutiveStar AS
    SELECT MS.name,MS.address,MS.gender,MS.birthdate,ME.cert#,ME.netWorth
    FROM MovieStar MS, MovieExec ME WHERE MS.name = ME.name;
```

Exercise 8.1.2 — Querying Views (a, b, c)

```
-- (a) Female individuals who are both stars and executives
SELECT name FROM ExecutiveStar WHERE gender = 'F';

-- (b) Executives who are studio presidents AND worth >= $10M
SELECT name FROM RichExec
WHERE cert# IN (SELECT cert# FROM StudioPres);

-- (c) Studio presidents who are also stars AND worth >= $50M
SELECT SP.name
FROM StudioPres SP, ExecutiveStar ES
WHERE SP.cert# = ES.cert#
  AND ES.netWorth >= 50000000;
```

Exercise 8.2.1 — Which Views Are Updatable?

View	Updatable?	Reason
RichExec	YES	Single relation (MovieExec), simple WHERE, no joins
StudioPres	NO	WHERE clause contains a subquery that references the base relation (Studio) — violates updatability rule
ExecutiveStar	NO	JOIN of two relations (MovieStar and MovieExec) in FROM clause — two-relation joins are not updatable

Exercise 8.3.1 — Declaring Indexes (a, b, c)

```
-- (a) Index on studioName of Movies:
CREATE INDEX StudioIndex ON Movies(studioName);

-- (b) Index on address of MovieExec:
CREATE INDEX ExecAddressIndex ON MovieExec(address);

-- (c) Composite index on genre AND length of Movies:
CREATE INDEX GenreLengthIndex ON Movies(genre, length);
```

End of Chapter 8

Views and Indexes

Topics Covered: Virtual Views · Updatable Views · Instead-Of Triggers · Indexes · Index Selection · Materialized Views